



AmigaED 4.0 Quick Manual

Kurzanleitung & Schnellreferenz

AmigaED ist eine plattformübergreifende C/C++- und m68k-Assembler-IDE für die klassische Amiga-Entwicklung.

Inhaltsverzeichnis

1. Einführung
2. Zwei Wege, deinen Code zu bauen
3. Die Toolbar im Überblick
4. Modus 1: Ad-hoc-Einzeldatei-Kompilierung
5. Modus 2: Projektgesteuerte Kompilierung
6. Schnellstart: Von der Vorlage zum sauberen Build
7. Der Einstellungs-Editor (Prefs)
8. Das Kontextmenü des Editors
9. Das Compiler-Output-Fenster
10. Empfohlene Compiler- & Linker-Schalter
11. Empfehlungen für Amiga-C-Programmierer

Einführung



AmigaED

Zwei Wege zum Compilieren

Eine kurze, bebilderte Anleitung zu Ad-hoc-Einzeldatei-Kompilierung, projektgesteuerter Kompilierung und dem Einstellungs-Editor.

AmigaED ist eine plattformübergreifende C/C++- und m68k-Assembler-IDE für die klassische Amiga-Entwicklung.

Hinweis: Die Bildschirm-Beschriftungen in dieser Anleitung sind auf Englisch (AmigaEDs Standardsprache) abgebildet. Die Anwendung selbst besitzt aber auch eine deutsche Oberfläche (View → GUI Language).

Zwei Wege, deinen Code zu bauen

AmigaED unterstützt zwei unabhängige Wege, um aus deinem Quellcode ein lauffähiges AmigaOS-Programm zu machen. Beide stehen immer gleichzeitig zur Verfügung – nutze einfach den, der gerade zu deiner Aufgabe passt.

1. Ad-hoc-Einzeldatei-Kompilierung – eine einzelne Quelldatei öffnen und direkt compilieren, ganz ohne Projekt-Einrichtung. Ideal für einen schnellen Test, ein Einzelwerkzeug oder zum Ausprobieren einer Idee.

2. Projektgesteuerte Kompilierung – mehrere Dateien (Quellcode, Header, Assembler, Icons, ...) in einem Projekt zusammenfassen. AmigaED verfolgt sie, erzeugt automatisch Makefiles und baut alles mit einem Klick. Ideal für alles mit mehr als einer Datei, eine GUI (ReAction/MUI), oder Programme, an denen du später weiterarbeiten willst.

	Ad-hoc	Projekt
Geeignet für	Schnelle Einzeldatei-Tests	Mehrdatei-/GUI-Programme
Einrichtung nötig	Keine – Datei einfach öffnen	Projekt anlegen oder importieren
Mehrere Dateien verknüpft	Nein	Ja
Makefiles	Werden nicht erzeugt	Automatisch (gcc, vbcc, SAS/C)
Bleibt zwischen Sitzungen erhalten	Nein	Ja (.aep-Projektdatei)

Die Toolbar im Überblick

Jede Toolbar-Schaltfläche entspricht exakt einem Menüeintrag – die Toolbar ist reine Abkürzung für die am häufigsten gebrauchten Einträge, keine eigenständige Funktion.



#	Schaltfläche	Entspricht Menü	Funktion
---	--------------	-----------------	----------

1	New	File → New	Öffnet einen neuen, leeren Editor-Tab.
2	Open	File → Open...	Öffnet eine bestehende Datei in einem neuen Tab.
3	Save	File → Save	Speichert den aktuellen Tab.
4	Save As	File → Save As...	Speichert den aktuellen Tab unter neuem Namen/Ort.
5	Print	File → Print file...	Druckt die aktuelle Datei.
6	Undo	Edit → Undo	Macht die letzte Änderung rückgängig.
7	Redo	Edit → Redo	Stellt die zuletzt rückgängig gemachte Änderung wieder her.
8	Cut	Edit → Cut	Schneidet die aktuelle Auswahl aus.
9	Copy	Edit → Copy	Kopiert die aktuelle Auswahl.
10	Paste	Edit → Paste	Fügt den Zwischenablage-Inhalt ein.
11	Suchen	(Kontextmenü) Search and Replace...	Öffnet das Suchen/Ersetzen-Panel für den aktuellen Tab.
12	Goto Line	Navigation → Goto Line...	Springt direkt zu einer angegebenen Zeilennummer.
13	Matching bracket	Navigation → Goto matching bracket	Springt zur zur Cursor-Position passenden Klammer.
14	Compile	Build → Compile...	Ad-hoc-Einzeldatei-Kompilierung (Modus 1) – siehe unten.
15	Build Project	Build → Build Project	Projekt-Build (Modus 2) – siehe unten.
16	Clean Project	Build → Clean Project	Entfernt die Build-Artefakte des aktuellen Projekts (Objektdateien, Executable, Icon).
17	Open Shell	Build → Open Shell	Öffnet die Standard-Kommandozeile des Systems, gestartet im Ordner des aktuellen Projekts (oder im konfigurierten „Projects root“, falls kein Projekt geladen ist).
18	Start Emulator	Tools → Emulator → Start default Workbench in UAE...	Startet den konfigurierten UAE-Emulator.
19	Stop Emulator	Tools → Stop running Emulation...	Beendet die laufende Emulator-Instanz.
20	Exit	File → Exit	Beendet AmigaED.

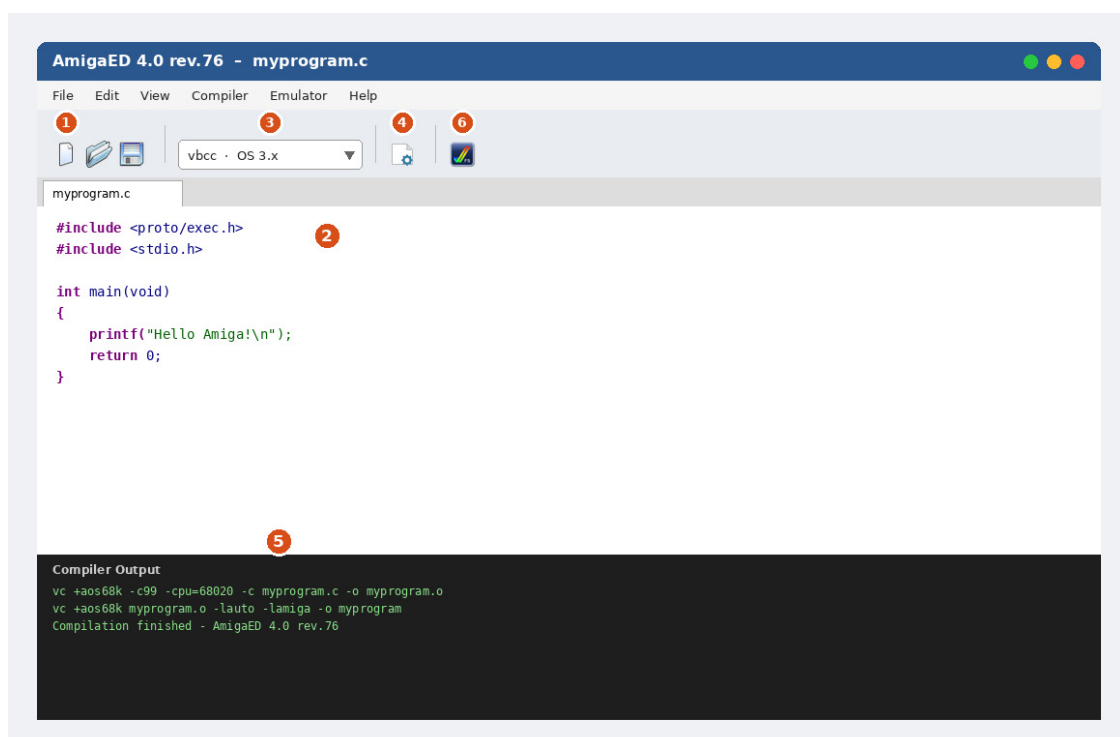
Warnhinweis: Open Shell öffnet eine uneingeschränkte Kommandozeile direkt im Projektordner – einschließlich

der Makefiles und sämtlicher Dateien, die ein Build liest oder erzeugt. Werden Dateien dort von Hand bearbeitet, umbenannt oder gelöscht, außerhalb der eigenen Aktionen von AmigaED, kann der Projektzustand nicht mehr zu dem passen, was AmigaED erwartet – der nächste Build Project- oder Clean Project-Lauf kann dadurch fehlschlagen oder sich unerwartet verhalten. Die Shell ist als Werkzeug für erfahrene Nutzer gedacht, nicht als Teil des normalen Bearbeiten-und-Bauen-Zyklus.

Hinweis: Start/Stop Emulator grauen sich automatisch gegenseitig ein bzw. aus, je nachdem, ob gerade eine Emulator-Instanz läuft – auch eine, die AmigaED selbst nicht gestartet hat (z. B. bewusst aus einer früheren Sitzung offen gelassen, oder ganz unabhängig von AmigaED gestartet).

Modus 1: Ad-hoc-Einzeldatei-Kompilierung

Nutze diesen Modus, wenn du nur eine einzelne Quelldatei schreiben und testen willst – ohne Projekt, ohne mehrere verknüpfte Dateien, ohne vorherige Einrichtung.



Der Editor im Ad-hoc-Modus: Die Nummern entsprechen den Schritten unten.

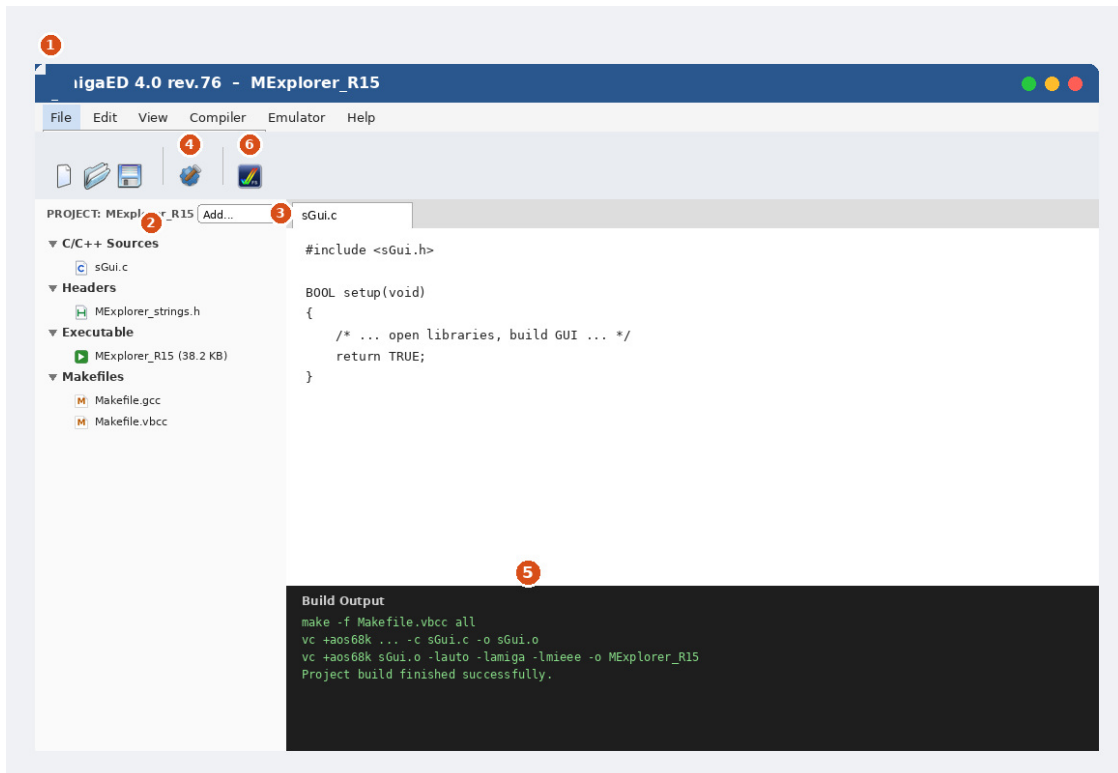
1. File → New (oder Open...), um eine einzelne .c-/.cpp-Datei zu erstellen oder zu öffnen. Es ist kein Projekt beteiligt.
2. Code im Editor-Tab schreiben oder bearbeiten – mit vollständigem Syntax-Highlighting.
3. In der Toolbar über das Compiler-Dropdown die gewünschte Toolchain und Ziel-OS auswählen (z. B. vbcc / OS 3.x, oder m68k-amigaos-gcc / OS 1.3).
4. Auf das Compile-Symbol in der Toolbar klicken (oder das Compiler-Menü nutzen), um nur diese Datei zu bauen.
5. Im Compiler-Output-Panel unten nachsehen – dort stehen die genauen Compiler-/Linker-Befehle und ob der Build erfolgreich war.
6. Optional auf Start Emulator klicken, um UAE zu starten und das frisch gebaute Programm gleich

auszuprobieren.

Hinweis: Die Ad-hoc-Kompilierung baut und linkt immer nur die eine geöffnete Datei. Falls dein Programm mehr als eine Quelldatei braucht, wechsele stattdessen zur projektgesteuerten Kompilierung (siehe nächstes Kapitel).

Modus 2: Projektgesteuerte Kompilierung

Nutze diesen Modus für alles mit mehr als einer Quelldatei, für eine GUI (ReAction oder MUI), oder für Programme, die du über längere Zeit weiterbauen und pflegen willst.



Das Projekt-Panel und die Toolbar: Die Nummern entsprechen den Schritten unten.

1. File → New Project..., um mit einer Vorlage zu starten (Empty C, AmigaShell, AmigaOS 1.3, AmigaOS 3.x, ReAction oder MUI), oder File → Import existing Project..., um einen Ordner mit Quelldateien einzubinden, den AmigaED noch nicht kennt.
2. Das Projekt-Panel links zeigt alle Dateien, automatisch in Kategorien sortiert – C/C++-Quellen, Header, Assembler-Quellen, AmigaGuide, Installer-Skripte, Executables, Sonstige Dateien, sowie die automatisch erzeugten Makefiles.
3. Jederzeit weitere Dateien über Add files to Project... hinzufügen, oder sie einfach per Drag & Drop auf das Projekt-Panel ziehen.
4. Auf Build Project klicken. AmigaED erzeugt die Makefiles für jede konfigurierte Toolchain (neu) und startet den Build.
5. Das Build-Output-Panel beobachten. Bei Erfolg erscheint das gebaute Programm im Projekt-Panel in der Kategorie Executable, zusammen mit seiner Dateigröße.
6. Auf Start Emulator klicken, um UAE zu starten und das Programm zu testen.

Hinweis: Die Makefiles eines Projekts werden automatisch neu erzeugt, sobald eine Datei hinzugefügt oder entfernt wird – und zusätzlich unmittelbar vor jedem Build Project, sodass auch reine Inhalts-Änderungen (z. B. nachträglich ergänzte Gleitkommazahlen in einer bereits erfassten Datei) zuverlässig erkannt werden, nicht nur Änderungen an der Dateiliste. Es werden gleichzeitig sowohl vbcc- als auch gcc/g++-Makefiles erzeugt, dazu ein SAS/C-Makefile für den späteren Bau auf einem echten Amiga.

Hinweis: Build Project speichert außerdem zuerst jede geöffnete Projektdatei mit ungespeicherten Änderungen – auch eine von Hand bearbeitete Makefile, nicht nur Quelldateien – da sowohl die obige Makefile-Regenerierung als auch der Compiler selbst immer direkt von der Platte lesen. Ob das automatisch geschieht oder vorher nachgefragt wird, steuert Prefs → Project → “Save Project Files Automatically” (siehe unten).

Ein Projekt schließen

File → Close Project schließt alle zum aktuellen Projekt gehörenden Tabs (mit Speicherabfrage bei ungespeicherten Änderungen – genau wie beim Wechsel zu einem anderen Projekt) und kehrt danach in den Zustand “kein Projekt geladen” zurück. Wird eine Speicherabfrage abgebrochen, bleiben Projekt und alle Tabs unverändert offen.

Automatische Programm-Icons

Ist Prefs → Misc → “create icon” aktiviert, schreibt AmigaED sein eigenes, eingebautes, mehrfarbiges Amiga-Tool-Icon direkt neben das kompilierte Programm – sowohl beim Projekt-Build als auch bei der Ad-hoc-Einzeldatei-Kompilierung. Es muss nirgends eine Standard-Icon-Datei konfiguriert werden. Die Stack-Größe des Icons wird automatisch passend zum Ziel gesetzt: 4096 Bytes für AmigaOS 1.3, 8192 für AmigaOS 3.x, 16000 für ein ReAction-Projekt und 34000 für ein MUI-Projekt.

Ein Projekt bereinigen

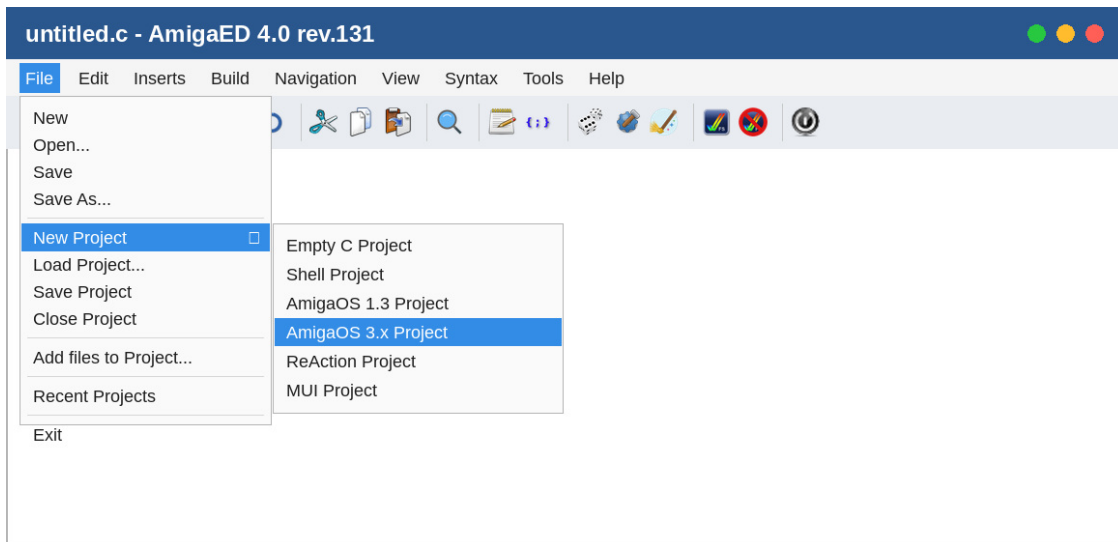
Build → Clean Project entfernt die Build-Artefakte des aktuellen Projekts – jede Objektdatei, das kompilierte Programm sowie dessen Icon – indem jede Datei einzeln und direkt gelöscht wird; jede entfernte Datei wird im Compiler-Output-Fenster aufgelistet. Dabei wird kein “make clean” über eine Shell aufgerufen.

Schnellstart: Von der Vorlage zum sauberen Build

Ein konkreter, bebildeter Durchlauf des oben beschriebenen Modus 2, von Anfang bis Ende: ein neues Projekt aus einer Vorlage erstellen, bauen, und wieder bereinigen.

1. Projekt anlegen

File → New Project bietet sechs Vorlagen: Empty C Project, Shell Project, AmigaOS 1.3 Project, AmigaOS 3.x Project, ReAction Project und MUI Project. Die ReAction- und MUI-Vorlagen bauen bereits ein kleines, funktionierendes Fenster mit File-Menü (About/Quit) – ein echter Startpunkt, keine leere Hülle.

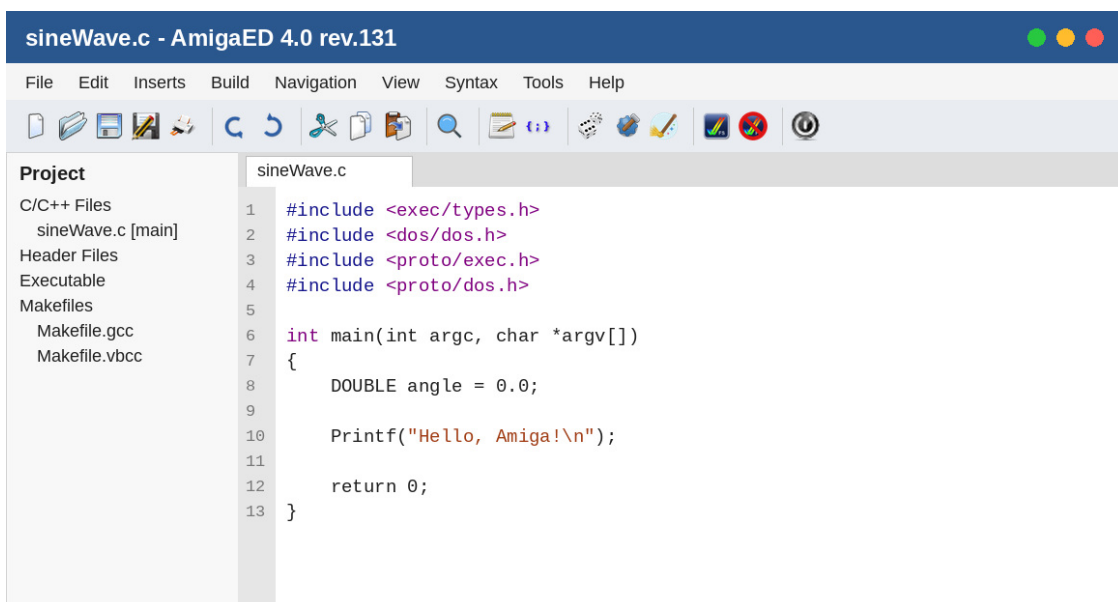


Auswahl von "AmigaOS 3.x Project" im New-Project-Untermenü.

Nach der Vorlagenauswahl fragt AmigaED nach einem Zielverzeichnis und einem Projektnamen, und bestätigt anschließend (bei den Vorlagen AmigaOS 1.3/3.x, ReAction und MUI) die Standard-Compiler-/Linker-Schalter für dieses Ziel, bevor irgendetwas angelegt wird – dieselben Schalter, die weiter unten in der Tabelle stehen, bereits vorausgefüllt und direkt anpassbar.

2. Code schreiben

Die erzeugte Hauptdatei öffnet sich automatisch, bereits mit den passenden Amiga-NDK-Headern für die gewählte Vorlage. Das Projekt-Panel links zeigt die Datei, die automatisch erzeugten Makefiles, sowie die (noch leere) Executable-Kategorie.



Ein frisch erstelltes AmigaOS-3.x-Projekt, bereit zur Bearbeitung.

3. Bauen

Auf das Build-Project-Symbol in der Toolbar klicken (oder Build → Build Project). AmigaED erzeugt die Makefiles frisch neu und ruft dann den aktuell gewählten Compiler auf (VBCC oder GCC/G++, wählbar in der Statusleiste). Verwendet der Code `float/double` – oder die auf Amiga typischen

großgeschriebenen Typedefs `FLOAT/DOUBLE`, beide werden erkannt – wird die passende Mathe-Bibliothek automatisch mitgelinkt; nichts muss von Hand konfiguriert werden.

Ein erfolgreicher Build, gemeldet im Compiler-Output-Fenster.

Siehe „Das Compiler-Output-Fenster“ weiter unten, um direkt von einer Fehler- oder Warnzeile zur betroffenen Quelltextzeile zu springen.

4. Aufräumen

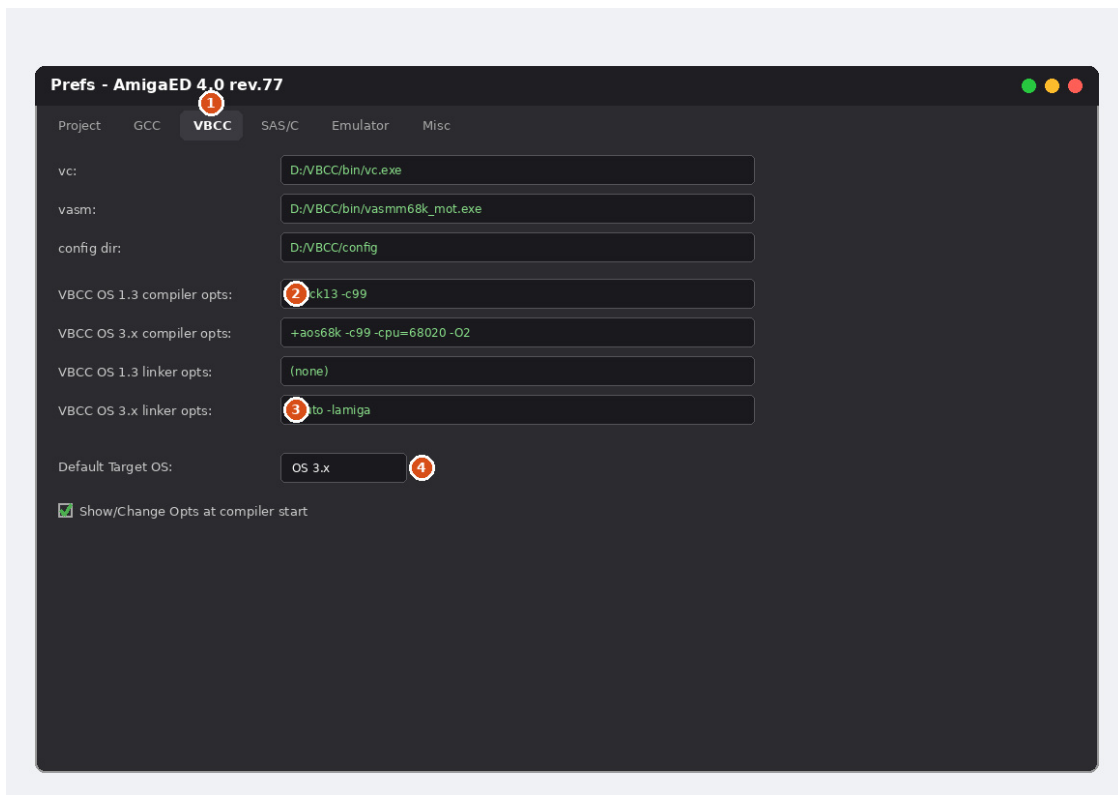
Auf das Clean-Project-Symbol in der Toolbar klicken (oder Build → Clean Project), um die Build-Artefakte wieder zu entfernen – jede Objektdaten, das kompilierte Programm sowie dessen Icon – jede Datei einzeln gelöscht und im Compiler-Output-Fenster aufgelistet.

Ausgabe von Clean Project: Jede entfernte Datei wird namentlich aufgeführt.

Hinweis: Deine Quelldateien, die .aep-Projektdatei und die Makefiles selbst werden von Clean Project nie angefasst – nur die Dateien, die ein Build tatsächlich erzeugt.

Der Einstellungs-Editor (Prefs)

File → Prefs... öffnet AmigaEDs Einstellungen, gegliedert in Reiter: Project (Autor/E-Mail/Website, Standard-Projektordner), GCC, VBCC und SAS/C (Pfade zu jeder Toolchain sowie deren Compiler-/Linker-Optionen – jeweils getrennte Felder für AmigaOS-1.3- und AmigaOS-3.x-Ziele), Emulator (Pfad zu deiner UAE-Programmdatei, sowie getrennte OS-1.3-/3.x-Konfigurationen) und Misc (Anwendungsstil/Theme, ob ein Programm-Icon erzeugt werden soll, sowie weiteres Editor-Verhalten).



Der VBCC-Reiter als Beispiel – GCC und SAS/C folgen demselben Muster.

1. Die Reiter oben wechseln zwischen Project-, GCC-, VBCC-, SAS/C-, Emulator- und Misc-Einstellungen.
2. Compiler-Optionen werden getrennt für AmigaOS 1.3 und AmigaOS 3.x vorgehalten – die beiden Ziele brauchen üblicherweise unterschiedliche Schalter (siehe Tabellen unten).
3. Linker-Optionen sind ebenso pro Ziel-OS getrennt – hier wird automatisch eine Mathe-Bibliothek ergänzt, wenn dein Projekt Gleitkommazahlen verwendet (siehe Hinweis unten).
4. Default Target OS legt fest, welcher der beiden obigen Optionssätze verwendet wird, wenn du nicht ausdrücklich einen auswählst.

Automatische Gleitkommazahlen-Behandlung: Erkennt AmigaED irgendwo in den eigenen C/C++-Quellen deines Projekts float oder double – oder die auf Amiga typischen großgeschriebenen Typedefs FLOAT/DOUBLE, beide werden erkannt – wird automatisch die passende Mathe-Bibliothek in den erzeugten Makefiles ergänzt – -lm für gcc/g++, -lmieee für vbcc, und MATH=IEEE für SAS/C (nur falls noch kein MATH=-Modus gesetzt ist). Das musst du nicht selbst ergänzen.

Speichern vor einem Build

Prefs → Project → “Save Project Files Automatically“ (direkt unter “Projects root“) legt fest, was Build Project tut, wenn eine geöffnete Projektdatei – eine Quelldatei ebenso wie eine von Hand bearbeitete Makefile – ungespeicherte Änderungen hat: aktiviert wird alles sofort ohne Nachfrage gespeichert;

deaktiviert (Standard) wird einmalig nachgefragt, mit einer Liste aller betroffenen Dateien und der Wahl, sie zu speichern, trotzdem mit dem Stand von der Platte zu bauen, oder den Build ganz abubrechen.

Designs

Prefs → Misc → “Default application style“ (oder das Menü View → Theme, das die Änderung sofort anwendet, ganz ohne Prefs zu öffnen) listet jeden auf deiner Plattform verfügbaren nativen Stil, plus drei synthetische: **Dark**, **Workbench 1.3** (der klassische blaue Kickstart-1.3-Desktop-Look, direkt aus einem echten Screenshot abgetastet) und **Workbench 3.1** (der schlichtere, neutral-graue AmigaOS-3.x-Look, ebenfalls aus einem echten Screenshot abgetastet). View → Theme zeigt alle als eine einzige, sich gegenseitig ausschließende Liste – die Auswahl wirkt sofort und überall: die Anwendungsoberfläche, jeder offene Editor-Tab, sowie die Fehler-/Warnfarben im Compiler-Output-Fenster ziehen gemeinsam mit.

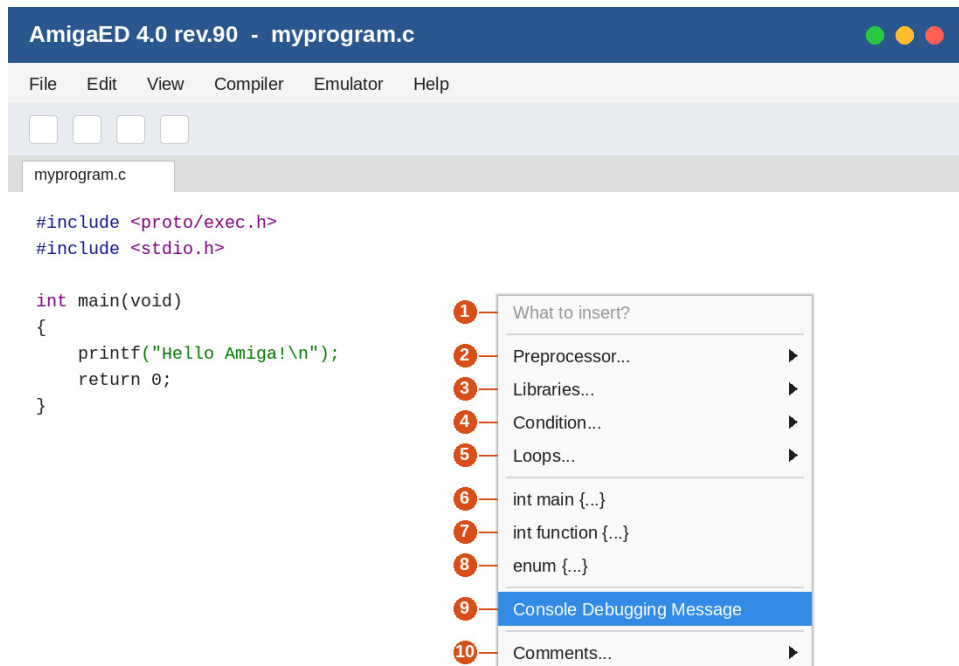
Der Emulator und laufende Instanzen

Wird AmigaED beendet, während der Emulator noch läuft, fragt es, ob der Emulator geöffnet bleiben oder mitbeendet werden soll – praktisch, da ein Emulator-Start echte Zeit kostet, die man sich für die nächste Sitzung sparen kann. Der Start des Emulators prüft außerdem über die eigene Prozessliste des Betriebssystems, ob bereits ein passender Emulator-Prozess läuft – egal ob von AmigaED selbst gestartet (z. B. so offen gelassen) oder nicht – und fragt nach, bevor eine zweite Instanz gestartet wird, statt das bedingungslos zu tun. Start/Stop Emulator (Toolbar und Tools-Menü) spiegeln automatisch wider, welcher dieser beiden Zustände gerade zutrifft.

Bekanntes Problem: FS-UAE unter WSL2/WSLg: wird die Linux-Version von FS-UAE innerhalb von WSL2 (Windows Subsystem for Linux) betrieben, ist das Einfangen der Maus unzuverlässig – sowohl der Host- als auch der emulierte Amiga-Mauszeiger können gleichzeitig sichtbar bleiben und bei Mausbewegung auseinanderdriften, unabhängig davon, wie FS-UAEs eigene Konfigurationsoptionen für das Maus-Greifen gesetzt sind. Das geht auf eine bekannte WSLg-Einschränkung bei relativer Mauseingabe zurück (siehe microsoft/wslg Issue #240), nicht auf einen Fehler in FS-UAE, AmigaED oder der eigenen Konfiguration – bestätigt dadurch, dass dieselbe FS-UAE-Konfiguration auf einer echten Linux-Maschine einwandfrei funktioniert. Wird FS-UAE unbedingt innerhalb von WSL2 benötigt, ist das Betreiben eines eigenen X-Servers auf der Windows-Seite (z. B. VcXsrv) statt WSLgs eigener GUI-Integration der vielversprechendere Workaround; andernfalls vermeidet der Betrieb von FS-UAE auf einer echten Linux-Maschine (oder nativ unter Windows, mit der Windows-Version von FS-UAE) das Problem vollständig.

Das Kontextmenü des Editors

Ein Rechtsklick irgendwo im Code-Editor öffnet ein Kontextmenü mit schnellen Code-Einfüge-Hilfen. Es spiegelt exakt das Hauptmenü Inserts (Menüleiste) wider – alles, was hier aufgeführt ist, findest du dort unter demselben Namen wieder.



Das Kontextmenü des Editors, geöffnet über einer Quelldatei: Die Nummern entsprechen den unten erklärten Einträgen.

1. **What to insert?** – eine deaktivierte Kopfzeile, kein anklickbarer Eintrag. Sie dient nur als Beschriftung des Menüs.
2. **Preprocessor...** – ein Untermenü mit Präprozessor-Einfügungen: `#include`, `#define`, `#ifdef`, `#ifndef`, `#if defined(...)`, **Amiga #include files** (ein fertiger Block der gängigsten Amiga-NDK-Header), **Identify Amiga compiler** (eine Kette aus `#if defined(...)`-Prüfungen, die eine Konstante `compiler_string` mit dem Namen des jeweils verwendeten Compilers erzeugt), sowie **Amiga C version string** (ein `$VER:-`Versionsstring).
3. **Libraries...** – `OpenLibrary()` und `CloseLibrary()` fügen je eine bearbeitbare `dummy.library`-Open/Close-Vorlage mit Fehlerbehandlung ein.
4. **Condition...** – Vorlagen für `if(..) {...}` und `if(..) {...} else {...}`.
5. **Loops...** – `while(...)` `{...}`, `for(...)` `{...}`, `do...{...}while(...)` sowie `switch(...)` (bereits mit einem `case dummy: break;`).
6. **int main {...}** – fügt eine vollständige `main()`-Funktion ein; deaktiviert, sobald die Datei bereits eine enthält.
7. **int function {...}** – fügt einen Funktionsprototyp plus passende Funktionsvorlage ein, bereit zum Ausschneiden und Einfügen an die richtige Stelle.
8. **enum {...}** – fügt eine vollständige C-Enumeration ein (`SomeEnum` mit drei Beispielwerten).
9. **Console Debugging Message** – fügt einen `if (myDebug) {...}`-Debug-Block ein; der Cursor landet direkt darin, bereit zum Tippen. Benötigt ein zuvor im Quelltext definiertes `#define myDebug TRUE` – das ist bereits am Anfang jeder "New Project"-Vorlage enthalten.
10. **Comments...** – **Fileheader comment...**, C-style Einzeil-/Mehrzeilkommentare, ein C++-Einzeilkommentar sowie ein C-artiger Trennlinien-Kommentar.

Hinweis: Die meisten Einträge fügen ihre Vorlage direkt an der Cursor-Position ein und positionieren den Cursor anschließend so, dass direkt weitergetippt werden kann – entweder in einer neuen Zeile nach dem eingefügten Block, oder (bei Console Debugging Message) direkt darin. Dieses Kontextmenü nutzt exakt dieselben Aktionen wie das Hauptmenü Inserts – beide sind daher immer synchron.

Das Compiler-Output-Fenster

Jede Kompilierung – Ad-hoc (Modus 1) oder Projekt (Modus 2), VBCC oder GCC/G++ – gibt ihre Ausgabe im Compiler-Output-Fenster unten aus, mit zwei Dingen, die das Durcharbeiten erleichtern: farblich markierte Diagnosen und Klick-zum-Springen.

Farblich markierte Fehler und Warnungen

Jede erkannte Fehlerzeile wird rot hervorgehoben, jede Warnung gelb/orange – fester Text auf getöntem Hintergrund, mit einem an das aktuelle Theme angepassten Farbschema (siehe Designs oben), damit es unabhängig vom gewählten Theme lesbar bleibt. Rein informative Zeilen (Echo des Compiler-Aufrufs, “In file included from...“, GCCs eigene Quelltext-Ausschnitt-/Zeigerzeilen) bleiben bewusst unmarkiert.

Klick zum Springen

Ein Klick auf eine Fehler- oder Warnzeile springt direkt zur betroffenen Datei und Zeile: Die Datei öffnet sich in einem Tab (oder wird einfach aktiviert, falls bereits offen), und der Cursor springt zur exakt gemeldeten Zeile und Spalte.

Compiler	Erkanntes Format
VBCC	error 9 in line 24 of "file.c": message
GCC/G++	file.c:24:5: error: message

Hinweis: Ein Fehler, der innerhalb einer erzeugten Zwischendatei gemeldet wird (z. B. wenn VBCC ein Problem auf Assembler-Ebene in der aus deiner .c-Datei erzeugten .asm-Datei meldet), springt weiterhin korrekt dorthin – zu genau der Zwischendatei, wie vom Compiler selbst gemeldet.

Empfohlene Compiler- & Linker-Schalter

Das sind AmigaEDs eigene, eingebaute Standardwerte – ein solider, gut erprobter Ausgangspunkt für jede Toolchain und jedes Ziel. Du kannst jederzeit eigene, projektspezifische Schalter in denselben Prefs-Feldern ergänzen.

m68k-amigaos-gcc (GCC/G++)

OS-1.3-Compiler	-Wall -O2 -mcrt=nix13	Übliche Warnungen, Optimierung, Bindung gegen libnix, gebaut für Kickstart 1.3.
-----------------	-----------------------	---

OS-3.x-Compiler	-Wall -O2 -noixemul	Übliche Warnungen, Optimierung, nutzt die AmigaOS-native C-Laufzeitumgebung statt ixemul.library.
OS-1.3-Linker	(keine)	Die OS-1.3-Vorlage ist reine Konsolenanwendung und braucht standardmäßig keine Zusatzbibliotheken.
OS-3.x-Linker	- lamiga	Für Workbench-fähige Programme (ReAction/MUI) nötig – wird von AmigaEDs OS-3.x-/ReAction-/MUI-Vorlagen automatisch ergänzt.

vbcc (vc)

OS-1.3-Compiler	+kick13 -c99	Benannte vbcc-Konfiguration für Kickstart 1.3, C99-Sprachmodus.
OS-3.x-Compiler	+aos68k -c99	Benannte vbcc-Konfiguration für AmigaOS 2.0+/3.x, C99-Sprachmodus.
OS-1.3-Linker	(keine)	Die reine Konsolen-OS-1.3-Vorlage braucht standardmäßig keine Zusatzbibliotheken.
OS-3.x-Linker	- lauto - lamiga	vbccs eigenes Äquivalent zu amiga.lib, nötig für Workbench-fähige (ReAction/MUI) Programme.

Optionale Ergänzungen, die viele Nutzer für einen Release-Build hinzufügen, sobald der Code sauber compiliert: -cpu=68020 (oder höher), -O2/-O3 (Optimierung), -size (Codegröße), -final (Laufzeitprüfungen abschalten). Diese sind bewusst nicht Teil von AmigaEDs eigenen Standardwerten, da sie Kompatibilität bzw. Debugbarkeit gegen Geschwindigkeit/Größe eintauschen.

SAS/C

SAS/C läuft nur auf einem echten Amiga oder Emulator, AmigaED ruft es daher selbst nie auf – es erzeugt lediglich ein Makefile.sc zusätzlich zu den anderen, für die spätere, manuelle Nutzung dort. Das SCOPTS-Feld in den Prefs ist zunächst leer; AmigaED ergänzt trotzdem automatisch MATH=IEEE, falls dein Projekt das braucht.

Empfehlungen für Amiga-C-Programmierer

Eine kurze, handverlesene Liste an Literatur und Werkzeugen, die in der Amiga-C-/ReAction-/MUI-Entwicklergemeinde besonders empfohlen werden – nichts davon liegt AmigaED bei, aber alles passt gut dazu.

Literatur

Amiga ROM Kernel Reference Manual: Exec / Libraries and Devices / Includes and Autodocs

Die ursprüngliche Commodore-Amiga-Referenzreihe – nach wie vor die maßgebliche Quelle für Exec, Intuition und den Rest der klassischen System-Bibliotheken. Vergriffen; freie Scans sind auf archive.org archiviert.

Amiga ROM Kernel Reference Manual: AmigaDOS – Thomas Richter

Eine moderne, im RKRM-Stil gehaltene Referenz speziell für AmigaDOS/dos.library. Kostenloser Download bei Aminet; eine gedruckte Ausgabe war zeitweise auch über lookbehindyou.de erhältlich.

ROM Kernel Reference Manual: Changes & Additions – Camilla Boemann & Jason Stead

Ein fortlaufendes (WIP-)Werk, das alles abdeckt, was dem AmigaOS von Release 2 bis 3.2 hinzugefügt wurde. Kostenloses PDF von developer.amigaos3.net.

Erik Bartmanns Amiga Programmierbuch

C-Programmierung, Crosscompiling mit vbcc und ReAction, auf einem Amiga 4000 (bzw. Amiga Forever). Buch: 32 EUR, E-Book: 19,99 EUR, über bombini-verlag.de.

Erik Bartmanns Amiga 1200 Buch

Maschinensprache, C-Programmierung und Shell – 717 Seiten in 28 Kapiteln, mit Fokus auf den Amiga 1200. Buch: 64 EUR, E-Book: 29,99 EUR, über bombini-verlag.de.

Werkzeuge

Rebuild – Darren Coles

Ein ReAction-GUI-Builder (eine komplette Neuentwicklung des klassischen ClassMate), der C- oder Amiga-E-Code erzeugt. Public Domain. Quelle: github.com/dmcoles/ReBuild

Codecraft – Camilla Boemann

Codecraft ist eine leistungsstarke IDE für die native Softwareentwicklung auf dem Amiga. Codecraft macht es für dich als Entwickler einfach, Code zu schreiben und das daraus entstehende Programm zu bauen, auszuführen und zu debuggen. Und das alles ist in einer einzigen, einheitlichen Benutzeroberfläche griffbereit. Eine native IDE für AmigaOS 3.2.3 und neuer. Quelle: gitlab.com/boemann/codecraft, <http://boemann.dk/codecraft/>

MUIBuilder

Ein Oberflächen-Designer für die MUI- und Zune-Toolkits. GPLv3/LGPLv3. Quelle: sourceforge.net/projects/muibuilder

Preise, Links und Verfügbarkeit ändern sich mit der Zeit – sollte oben etwas nicht mehr erreichbar sein, findet eine Websuche nach Titel und Autor meist am schnellsten die aktuelle Quelle.